

Spinel x Raspberry Pi Pico 2で シンセサイザーを作ってみた

名古屋Ruby会議05 LT

いしがき楽器亭
りょう

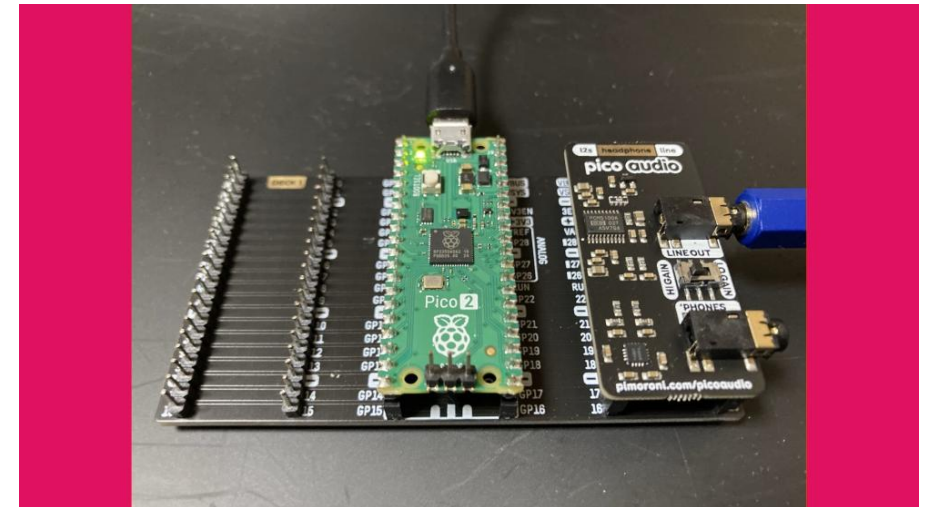
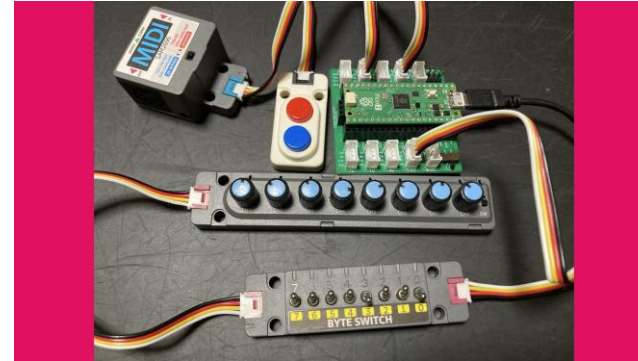
SPMS-1 (type-0) ライブ演奏

PicoRubyで作ったMIDI Controller
PRMC-1 (type-8)

で

Spinelで作ったMIDI Synthesizer
SPMS-1 (type-0)

を鳴らします

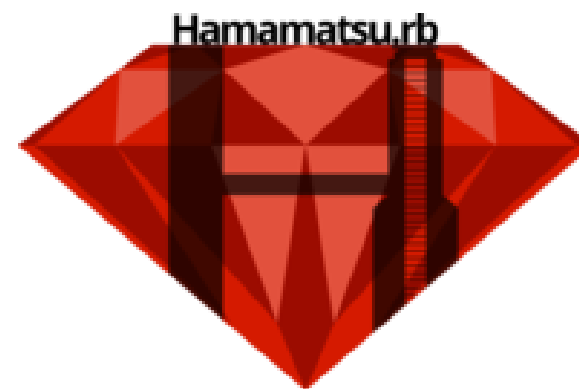


Thank you for listening!

ご清聴ありがとうございました！

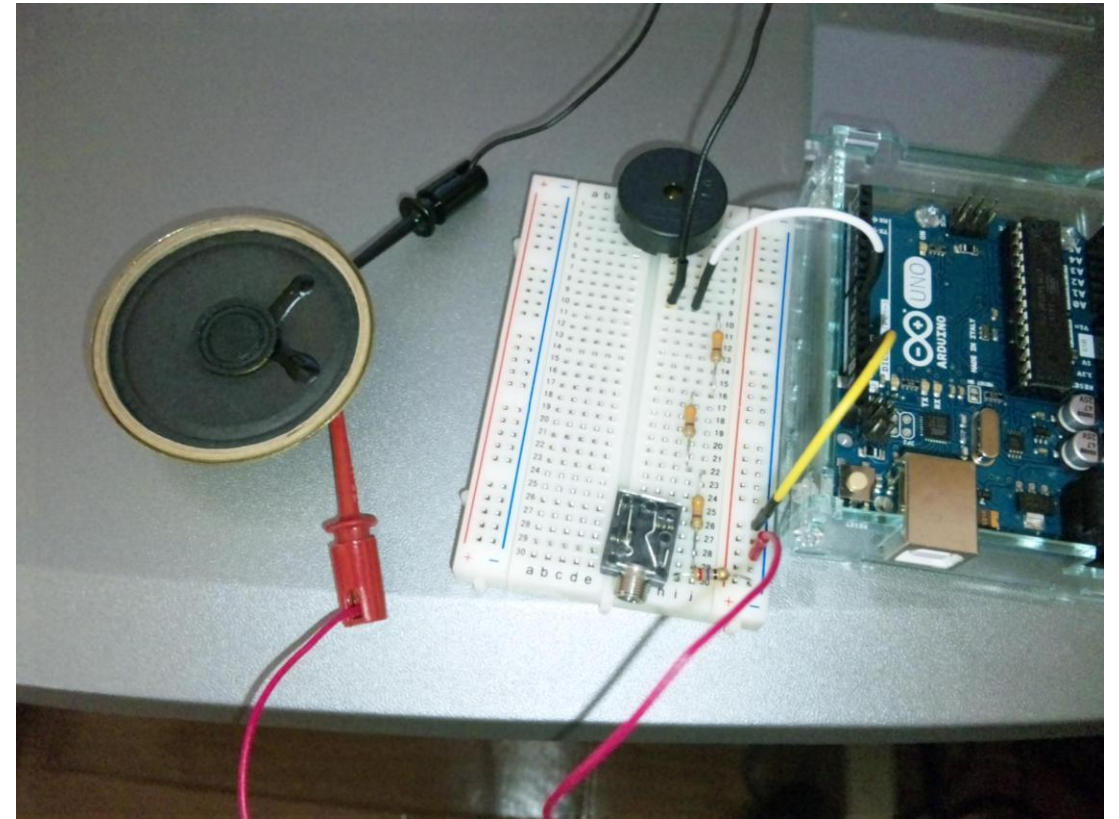
自己紹介

- Ryo Ishigaki (石垣 良)
 - <https://x.com/risgk>
- Hamamatsu.rb (ハママツルビー) メンバー
 - “Ruby誕生の地は物理的には浜松市”
- 組み込みソフトウェア開発者
 - プロジェクター、切削加工機、音響機器など
- 趣味で、マイコンで動くシンセサイザーなどの電子楽器を作っています
- 名古屋Ruby会議への前回の参加は03 (2017年)

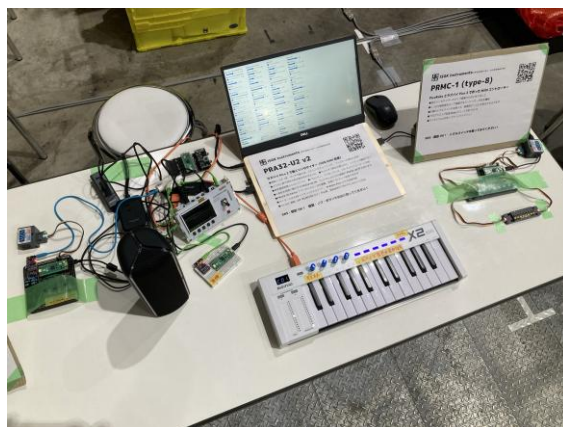
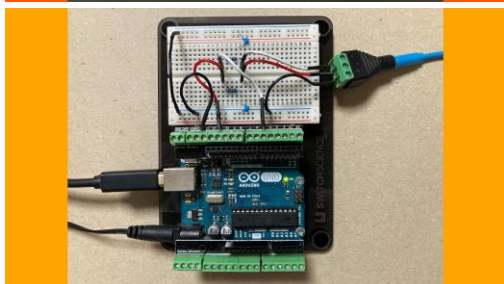
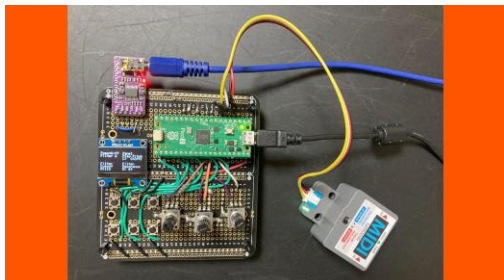


シンセサイザー作りの始まり

- 浜松は「楽器（作り）のまち」として知られています
- 私がシンセサイザー（シンセ）作りを始めたキッカケの一つは、浜松に引っ越したことです
- 2014年、私は最初のシンセサイザー **VRA8** のプロトタイプを **Ruby** で作り、**Arduino UNO Rev3** で動くようにC++に移植しました
 - 詳細：Ruby x Arduinoでシンセサイザーを作ってみた（2015年、浜松Ruby会議01）
<https://gist.github.com/risgk/0db52ea683530652d933>

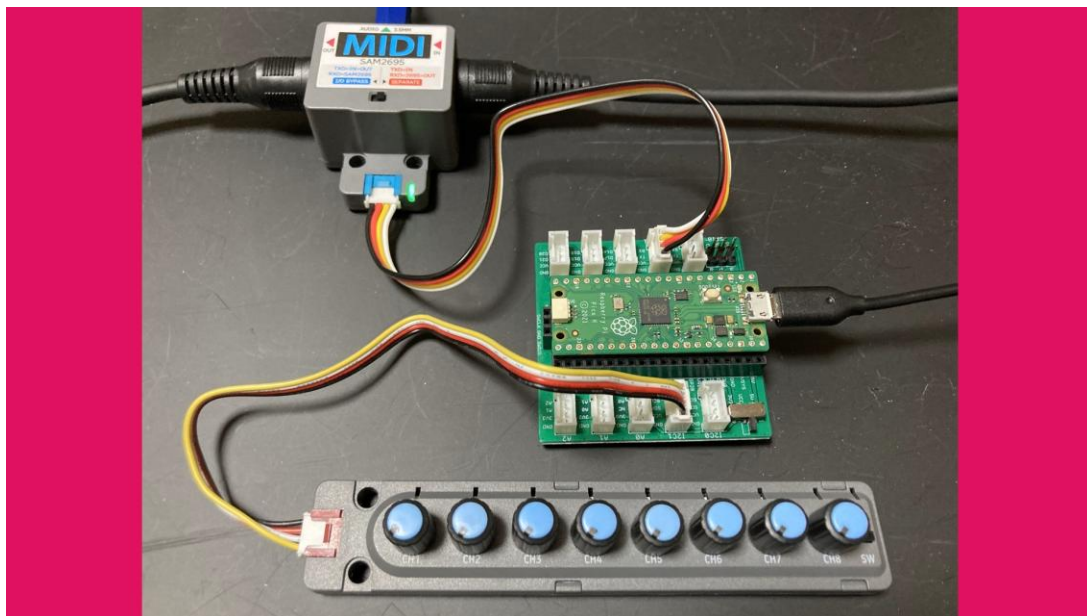


シンセサイザー作りと演奏



- ISGK Instruments 名義で、主にC++で年に1台ほどシンセサイザーを作り、Maker Faireなどのイベントで展示しています
 - **PRA32-U2** (Raspberry Pi Pico 2 用)
<https://github.com/risgk/digital-synth-pra32-u2>
- シンセサイザー製作記事を書いたことも
 - 『ラズベリー・パイPico/Pico W攻略本』
<https://interface.cqpub.co.jp/magazine/2023pico/>
- 少しだけ、ハードウェアシンセを使った演奏の「マシンライブ」を行っています
 - シンセカイリアル (Yasushi.Kさん主催)
 - RubyKaigi 2025 LT
 - HR Music Arena (RubyKaigi 2026)

MIDI Controller PRMC-1



- PicoRubyで作ったライブ用MIDIコントローラ、RubyKaigi 2025 LTで**type-0**を発表
 - https://github.com/risgk/midi_controller_prmc1
 - <https://speakerdeck.com/risgk/r2p2-rubykaigi-2025-lt>
 - やっと、mrubyファミリーを使った作品を作れました！
- RubyKaigi 2025 事後勉強会 otoでは、今後のテーマとして「Rubyのコードで音響合成するシンセをマイコンで動かす」をあげました
 - <https://speakerdeck.com/risgk/r2p2-rubykaigi-2025-shi-hou-mian-qiang-hui-oto>
 - Rubyで演奏するのもたのしいですが、私も、改めてRubyでゼロから音を作りたいになりました！（マイコンも使って！）

Raspberry Pi Pico 2

マイコンボード	Arduino UNO Rev3	Raspberry Pi Pico 2
発売年	2012年	2024年
価格	約3,000円（互換品は数百円～）	約1,000円
マイコン	ATmega328P	RP2350A
CPUコア	8ビット AVR 16 MHz 1コア	32ビット ARM Cortex-M33 150MHz 2コア
FPU（浮動小数点演算ユニット）	なし	あり（単精度、倍精度）
フラッシュメモリ、SRAM	32 KB、2 KB	4 MB、520 KB
使用可能なRuby処理系	基本的に無し	PicoRuby、mruby、mruby/c

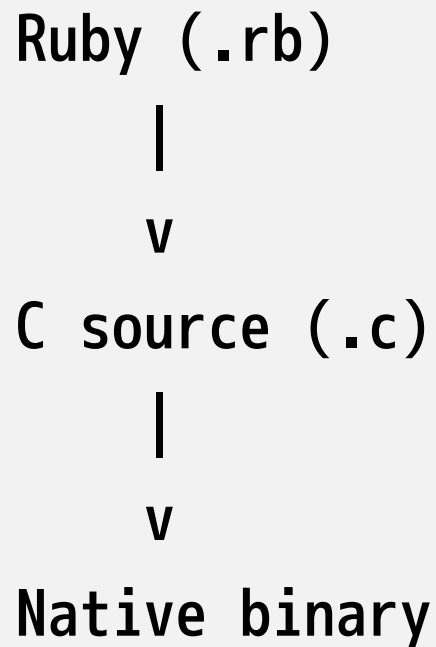
- Pico 2は現在最も広く使われているボードのひとつで、現在のPicoRubyの公式ボード
- Arduino UNO Rev3よりも遥かに強力！ 浮動小数点演算も高速！

試行錯誤

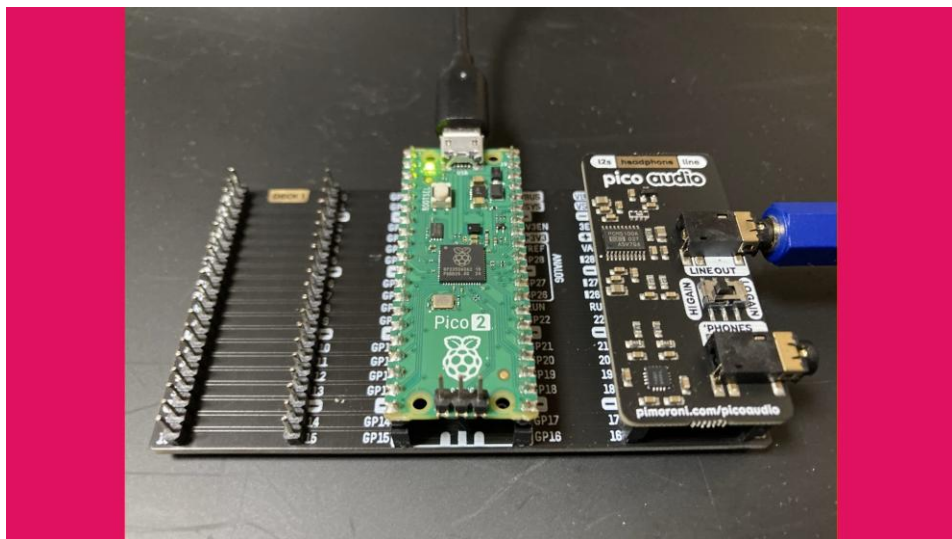
- **その1** : Raspberry Pi Pico 2とPicoRuby（当時のVMはmruby/c）を使って、Ruby音響合成するシンセサイザーを作ってみる
 - リアルタイムなデジタル信号処理（DSP）は厳しかったです…
 - 「高速UART出力で波形を作る」アイデアを検討しましたが、お蔵入り
- **その2** : PicoRubyデバイスから「MIDIで自由に制御できる」音響モジュラーシステムを（C++で）作ってみる
 - Web Audio APIなどが参考になります
 - 実現できそうですが、やりたかった「Rubyで音のプログラミング」からは遠ざかる感じが…

Spinel

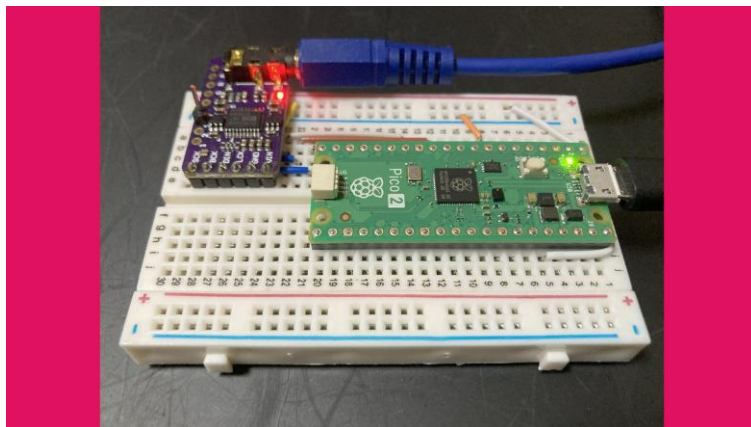
- **RubyKaigi 2026**最終日の4月24日に公開された、Rubyの**AOTコンパイラ**
 - <https://github.com/matz/spinel>
- Rubyのソースコードをスタンドアロンのネイティブ実行ファイルにコンパイル
- プログラム全体の型推論を行い、最適化されたCコードを生成することで、CRubyを大幅に上回る高速化を実現
 - Spinelが生成したCコードなら、マイコンでリアルタイム信号処理が可能なはず！
 - ランタイムはマイコン用に少し改造



MIDI Synthesizer SPMS-1 (type-0)



- SpinelとRaspberry Pi Pico 2で作った、減算方式の単音シンセサイザー
 - https://github.com/risgk/midi_synthesizer_spms1
 - バイナリファイル (UF2) も同梱
- サンプルレート48kHz、オーディオ出力24bit
 - 発音遅延時間 (レイテンシ) は約2.7ms
- USB MIDIで制御される音源モジュール
 - I2S (Inter-IC Sound) DACボードとして、Pimoroni Pico Audio Packを使用
 - 安価なGY-PCM5102も使用可能
- 信号処理コードは、すべてRubyで書きました！
 - MIDI入力やDAC出力には、C++を使用 (Arduino-Picoコア環境)
 - Spinelランタイムは、C言語を使用



MIDI (Musical Instrument Digital Interface)

- 電子楽器をつなぐ、40年以上使われている共通規格
- 「どう演奏するか」を数バイトのMIDIメッセージで送信
 - Note On/Off : ある高さの鍵盤をある速さで押した / 離れた
 - Control Change (CC) : 音色パラメータの調整値
- 物理層は31.25 kbaudのUARTで、今ではUSB MIDIも一般的

PicoRubyで作ったMIDI Controller
PRMC-1 (type-8)

↓ UART MIDI

USB MIDI Interface

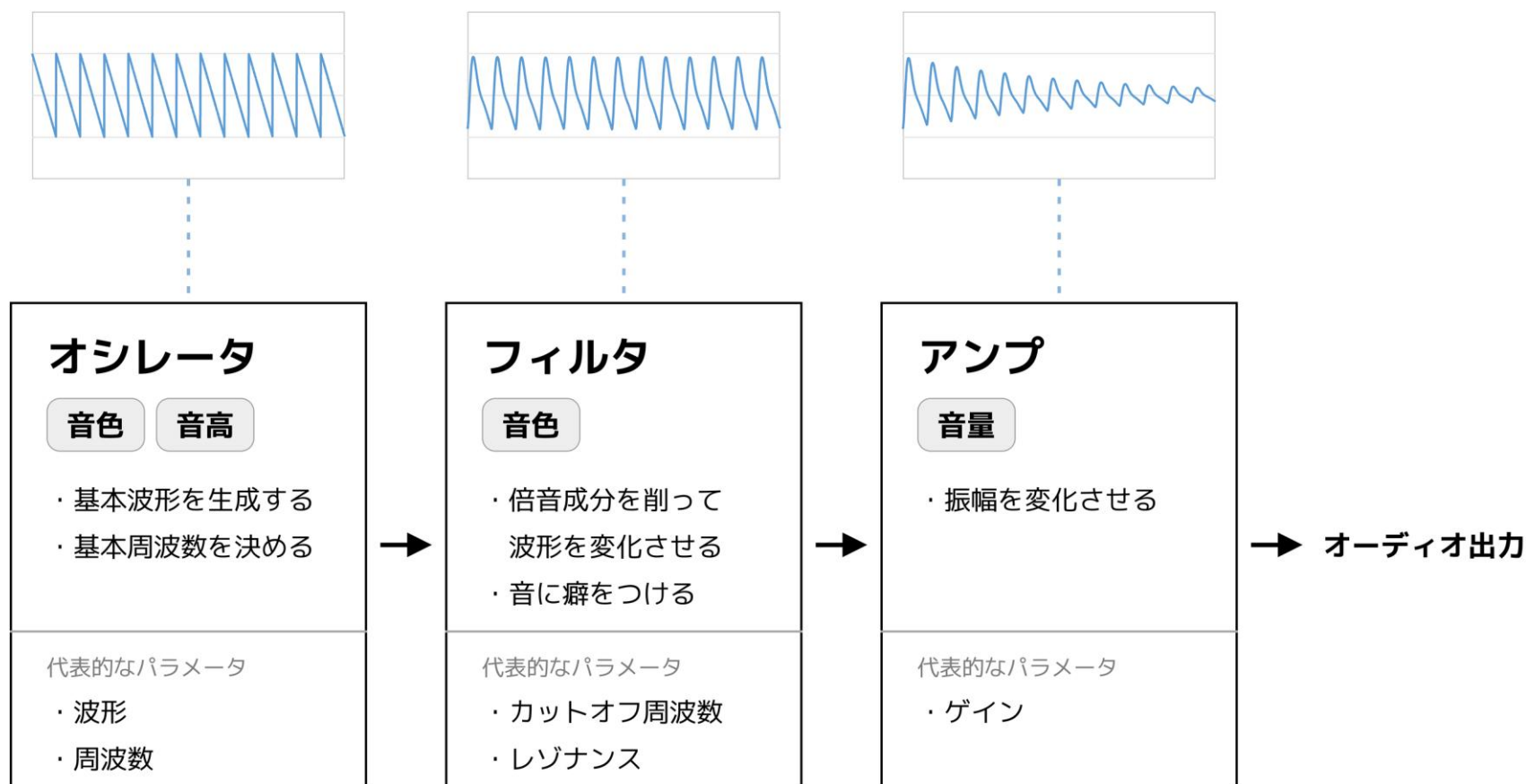
↓ USB MIDI

SPMS-1 (type-0) Editor -- Web App

↓ USB MIDI

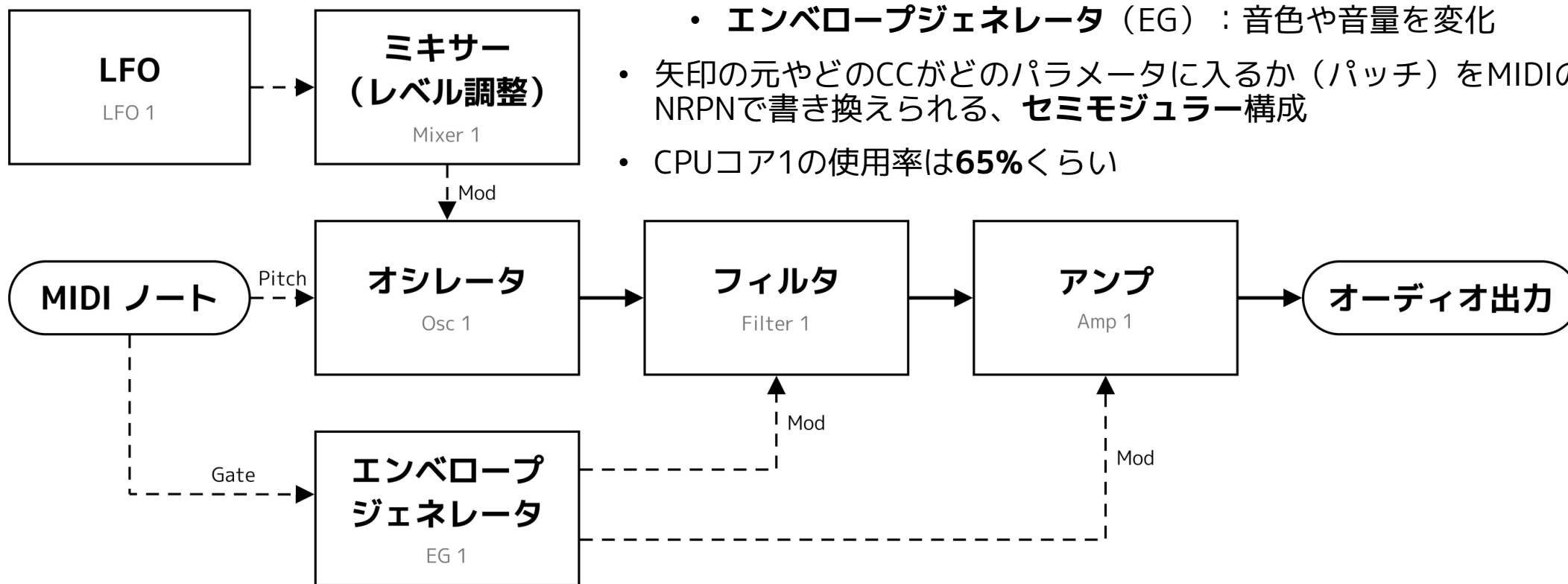
Spinelで作ったMIDI Synthesizer
SPMS-1 (type-0)

減算方式シンセサイザーの三大モジュール



SPMS-1のモジュール構成

- 制御信号でパラメータを時間変化させる仕組みを**モジュレーション**（変調）と呼びます
 - LFO**：低周波を出力、音高を周期変化（ビブラート）
 - エンベロープジェネレータ**（EG）：音色や音量を変化
- 矢印の元やどのCCがどのパラメータに入るか（パッチ）をMIDIのNRPNで書き換えられる、**セミモジュラー**構成
- CPUコア1の使用率は**65%**くらい



————> オーディオ信号 - - - -> 制御信号

spms1_type0.ino (Arduinoスケッチ) より

- ・最適化オプション指定、コードをSRAMに配置

```
#pragma GCC optimize ("O3")
...
#pragma GCC section text=".time_critical"
```

- ・CPUコア1 (SPMS-1のmain関数进行处理)

```
void setup1() {
}

void loop1() {
    Spms1_main(0, NULL);
}
```

- ・CPUコア0 (MIDI受信、デバッグプリント)

```
void setup() {
...
}

void loop() {
...
    USB_MIDI.read();
...
    SPMS1_DEBUG_PRINT_SERIAL.print(
        g_debug_measurement_min_us);
...
}
```

sp_runtime.h (改造版) より

```
#define main __attribute__((section(".time_critical"), flatten)) Spms1_main
#pragma GCC optimize("single-precision-constant")
#pragma GCC optimize ("O3")
...
#pragma GCC diagnostic ignored "-Wunused-parameter"
...
#define double float
#define sin(x) sinf(x)
#define cos(x) cosf(x)
...
//static inline mrb_int sp_process_ppid(void) {
//  return (mrb_int)getppid();
//}
```

- ・ 7月18日のコミット 5af61ae を改造
※コメントアウトやコード修正が
約10ファイルで約300行
- ・ Spinelが生成するmain関数をリネーム
- ・ コードをSRAMに配置
- ・ doubleを単精度floatに置き換え

spms1_main.rbより・その1

```
module Spms1
  module C
    ffi_func :set_midi_note_on_pitch, [:uint8, :uint8], :void
    ffi_func :get_midi_note_on_pitch, [:uint8], :uint8
    ...
    ffi_func :set_midi_cc_value, [:uint8, :uint8, :uint8], :void
    ffi_func :get_midi_cc_value, [:uint8, :uint8], :uint8
    ...
    ffi_func :start_audio, [], :void
    ffi_func :stop_audio, [], :void
    ffi_func :write_to_audio_buffer, [:float, :float], :void
    ffi_func :start_debug_measure, [], :void
    ffi_func :stop_debug_measure, [], :void
  end
end
```

・ FFI (Foreign Function Interface) で
Cの関数を呼びます

spms1_main.rbより・その2

```
signals = Array.new(SIGNALS_SIZE, 0.0)
```

```
...
```

```
  while i < AUDIO_BUFFER_WORDS
```

```
...
```

```
    while slot < MODULES_SIZE
```

```
      module_id = active_modules[slot]
```

```
...
```

```
      case module_id
```

```
...
```

```
        when MODULE_OSC_1
```

```
          signals[SIGNAL_OSC_1_OUTPUT] = osc_1.process(signals[source_osc_1_pitch], signals[source_osc_1_mod])
```

```
        when MODULE_FILTER_1
```

```
          signals[SIGNAL_FILTER_1_OUTPUT] = filter_1.process(signals[source_filter_1_audio], signals[source_filter_1_mod])
```

```
        when MODULE_AMP_1
```

```
          signals[SIGNAL_AMP_1_OUTPUT] = amp_1.process(signals[source_amp_1_audio], signals[source_amp_1_mod])
```

```
        end
```

- ・ Float値0.0を指定して配列生成することで、型推論が働き、GCの対象になるヒープメモリ割当を回避できます

※GCが作動したら、信号処理はほぼ破綻します！

- ・ 1/48000秒毎に、アクティブなモジュールを順番に処理
- ・ 1サンプル遅延しますが、
処理順が後のモジュールの出力信号も入力にできます
- ・ source_osc_1_mod などの設定をMIDIで変更できます

spms1_main.c (AOTコンパイル結果) より

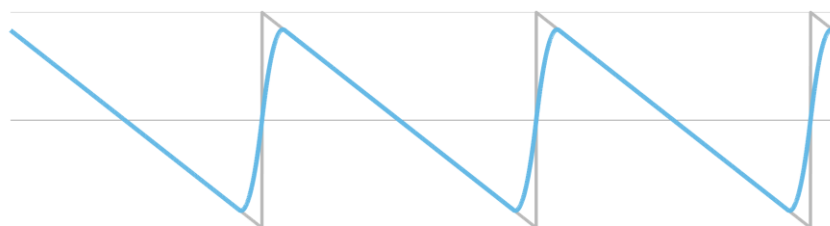
```
        else if ((_t312 == cst_MODULE_OSC_1)) {  
#line 620 "spms1_main.rb"  
        mrb_float _t314 = sp_FloatArray_get(lv_signals, lv_source_osc_1_pitch);  
        mrb_float _t315 = sp_FloatArray_get(lv_signals, lv_source_osc_1_mod);  
        sp_FloatArray_set(lv_signals, cst_SIGNAL_OSC_1_OUTPUT, sp_Osc_process((sp_Osc *)lv_osc_1, _t314, _t315));  
    }  
    else if ((_t312 == cst_MODULE_FILTER_1)) {  
...  
    }  
    else if ((_t312 == cst_MODULE_AMP_1)) {  
#line 624 "spms1_main.rb"  
        mrb_float _t318 = sp_FloatArray_get(lv_signals, lv_source_amp_1_audio);  
        mrb_float _t319 = sp_FloatArray_get(lv_signals, lv_source_amp_1_mod);  
        sp_FloatArray_set(lv_signals, cst_SIGNAL_AMP_1_OUTPUT, sp_Amp_process((sp_Amp *)lv_amp_1, _t318, _t319));  
    }
```

- ・ Rubyの配列アクセスでは長さチェックが入るので、
徹底的に最適化したCコードの性能には追いつけていません
- ・ Spinelを意識した最適化も、Claude CodeなどのAIに相談できます
※ Spinelのバージョンによってポイントは変わるでしょう

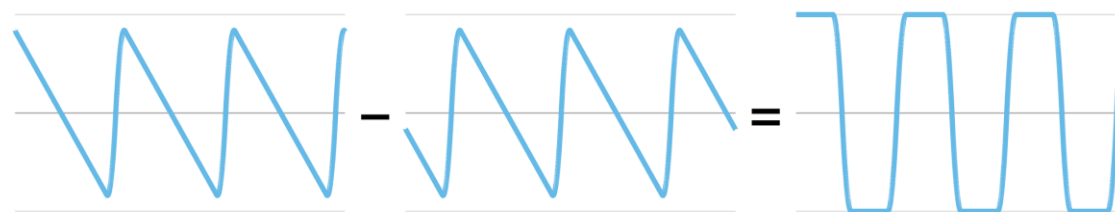
spms1_osc.rb（オシレータクラス）より

```
def process(pitch_input = 0.0, modulation_input = 0.0)
...
  naive_saw1 = -2.0 * @phase + 1.0
  blep1 = poly_blep(@phase, current_dt, current_dt_inv)
  saw1 = naive_saw1 + blep1
...
  output = saw1 - (saw2 * @current_waveform)
  @phase += current_dt
  @phase -= (@phase < 1.0) ? 0.0 : 1.0
...
end
```

- ・ 波形の角を丸めてエイリアスノイズを低減する
PolyBLEP法で、2個のノコギリ波を生成します
※このような対策を行わないと、
高い音でノイズが目立ちます！
- ・ ノコギリ波の減算で矩形波を作ります
- ・ 処理時間が安定するように、主な処理は
条件分岐なし（**ブランチレス**）実装にしています



— 補正なし — PolyBLEP



ノコギリ波 1

ノコギリ波 2
(半周期ずらし)

矩形波

spms1_filter.rb (フィルタクラス) より

```
def process(audio_input = 0.0, modulation_input = 0.0)
...
  if @sample_counter == 0
    update_coefficients
  end
...

  # Transposed Direct Form II (TDF-II) biquad implementation with soft clipping.
  audio_output = @z1 + @b0 * driven_input
  @z1 = soft_clip(@z2 + @b1 * driven_input - @a1 * audio_output)
  @z2 = soft_clip(@b2 * driven_input - @a2 * audio_output)
...

  clip_output(audio_output)
end
```

- ・ オーディオ処理でよく使われる、バイクアッド (**biquad**) フィルタ実装
- ・ さらに、Jatin Chowdhuryさんの**nonlinear biquad**手法で、フィードバック成分や出力をソフトクリップ (飽和) させています

<https://jatinchowdhury18.medium.com/complex-nonlinearities-episode-4-nonlinear-biquad-filters-ae6b3f23cb0e>

- ・ フィルタ係数の計算はサインやコサインを使っていて負荷が高いため、1/48000秒毎でなく1/12000秒毎に処理しています (**制御レート処理**)

まとめと今後

- Rubyコードを使った音のプログラミング、楽しかったです！
 - 私にとって初めての浮動小数点演算が中心のシンセでした
- SPMS-1の開発を続けます
 - type-1以降で機能追加することを考えています
 - パッチエディタ、Web Audio APIでの音出し環境なども作りたいです
 - Spinel改善のためのネタも集めていきたいです
- Ruby、Spinelでオーディオ処理することの意味は、プログラムやデバイスを作りながら考えていこうと思います

Enjoy Audio Programming!